

Progetto di Ingegneria Informatica

Checkpoint/Restart Fault Tolerance su STM32

Autore: Luca Aceti (Codice persona 10961427)

Tutor: Dott. Federico Reghenzani

Anno Accademico: 2025/2026

Repository GitHub: <https://github.com/acio404/PII-stm32-fault-tolerance-system>

Indice

1. Specifiche	3
1.1 Obiettivo.....	3
1.2 Setup sperimentale.....	3
1.3 Specifica.....	4
1.4 Vincoli operativi.....	5
2. Implementazione	6
2.1 Architettura checkpoint/restart	6
2.2 Persistenza su <i>EEPROM</i> e politica di validità.....	6
2.3 Procedura di restore	7
2.4 Benchmark applicativo (LU decomposition)	7
3. Test eseguiti	9
3.1 Cause di reset e corretto recupero dello stato	9
3.2 Reset con dump in <i>EEPROM</i> non completato.....	11
3.3 Reset relativi alla fase di risoluzione.....	12
3.4 Corretta cancellazione della firma	13
4. Analisi delle prestazioni e ottimizzazione hardware.....	14
5. Conclusioni e sviluppi futuri	15

1. Specifiche

1.1 Obiettivo

Lo scopo di questo progetto consiste nell'implementare un meccanismo di fault tolerance di tipo Checkpoint/Restart (C/R) su piattaforma STM32 in ambiente bare-metal (senza l'ausilio di un RTOS). Nei sistemi di calcolo embedded, in particolare in quelli safety-critical, i guasti dovuti a interruzioni di alimentazione improvvise o a reset causati da disturbi elettromagnetici sono una minaccia concreta per l'integrità del calcolo. Senza sistemi di tolleranza ai guasti, un arresto improvviso costringerebbe a ricominciare l'intera computazione da zero. Questo comporterebbe:

- Spreco di tempo di calcolo;
- Consumo inefficiente di energia;
- Rischio di non giungere mai al completamento dell'operazione se il tempo medio tra i guasti (MTTF) è inferiore alla durata del calcolo completo.

L'obiettivo di questo sistema è dunque quello di salvare periodicamente i dati necessari alla ripartenza in una memoria non volatile, cosicché in seguito ad un reset o ad una mancanza di alimentazione si possa ripartire dall'ultimo stato significativo tramite una procedura di ripristino della RAM e dei registri della CPU.

1.2 Setup sperimentale

La sperimentazione è stata condotta sulla scheda STM32 NUCLEO-L152RE, una scheda di sviluppo in formato Nucleo-64 basata sul microcontrollore STM32L152RET6. La MCU adotta un'architettura ARM Cortex-M3 a 32 bit con frequenza massima di 32 MHz; questa scelta rende la piattaforma adatta a valutare un meccanismo di Checkpoint/Restart in ambiente bare-metal su una famiglia di microcontrollori largamente diffusa in ambito embedded.

Dal punto di vista delle risorse on-chip, la piattaforma mette a disposizione 512 KB di Flash, 80 KB di SRAM e 16 KB di EEPROM interna. Quest'ultimo elemento è particolarmente rilevante per il progetto, poiché consente di memorizzare i checkpoint senza ricorrere a memorie esterne; inoltre, durante i test sono stati sfruttati la UART per il monitoraggio seriale e il pulsante di reset NRST per la fault injection manuale. Come si può vedere nella figura 1.2, la board adotta il classico formato Nucleo-64 con ST-LINK integrato.

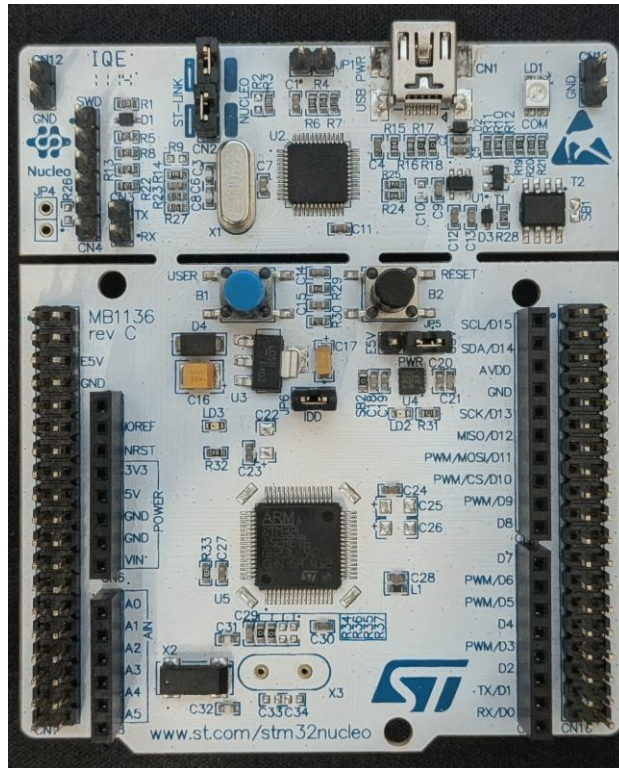


Figura 1.2. Board STM32 NUCLEO L152-RE

1.3 Specifica

Il sistema deve salvare internamente il contesto architetturale e applicativo minimo necessario alla ripartenza. Il trigger per l'acquisizione del contesto avviene via software, sfruttando l'eccezione standard ARM *PendSV*. Nello specifico, la struttura dati serializzata in memoria non volatile comprende:

- Una *Magic Word* per indicare la presenza di un checkpoint valido e prevenire falsi ripristini;
- L'indice sequenziale del checkpoint più recente;
- La dimensione in byte dello Stack attivo al momento dell'interruzione;
- Il *Main Stack Pointer* (MSP) corrente;
- I registri pushati automaticamente dall'hardware nell'*Exception Frame* al momento dell'interrupt (R0-R3, R12, LR, PC, xPSR);
- I registri operativi non volatili salvati manualmente via Assembly (R4-R11);
- Il dump fisico della memoria Stack (dall'MSP fino all'indirizzo base *_estack*), che garantisce il salvataggio intrinseco di tutte le variabili e strutture dati locali.

1.4 Vincoli operativi

La piattaforma su cui è stato sviluppato il meccanismo è una STM32 NUCLEO-L152RE, che ha permesso di utilizzare come memoria di persistenza la propria *EEPROM* interna. La progettazione del meccanismo sotto queste ipotesi presenta dei limiti a livello hardware, principalmente per due motivi:

- Latenza temporale: la scrittura in *EEPROM* è costosa (circa 4ms per word);
- La memoria è soggetta a usura, con un ciclo di vita limitato in termini di operazioni di scrittura/cancellazione.

Per adattarsi a questi vincoli e ridurre la finestra di vulnerabilità durante il salvataggio, è stato necessario implementare un meccanismo capace di calcolare a runtime l'effettivo ingombro dello Stack attivo ($dimensione_stack = _estack - MSP$). Questo approccio evita il dump indiscriminato dell'intera RAM, minimizzando i byte trasferiti in *EEPROM*, riducendo drasticamente il tempo di scrittura e preservando il ciclo di vita del componente.

2. Implementazione

2.1 Architettura checkpoint/restart

L'architettura del sistema ha richiesto l'utilizzo di funzioni in C, istruzioni assembly inline e configurazioni hardware del core ARM Cortex-M3. Inizialmente è stata definita la struttura *SystemState* con la funzione di mappare i dati e i registri principali:

```
typedef struct {
    uint32_t valid_checkpoint;
    uint32_t latest_checkpoint;
    uint32_t stack_size;
    uint32_t msp;
    uint32_t r0;
    uint32_t r1;
    uint32_t r2;
    uint32_t r3;
    uint32_t r12;
    uint32_t lr;
    uint32_t pc;
    uint32_t psr;
} SystemState;
```

I checkpoint vengono innescati chiamando un interrupt tramite *PendSV*, il processo si suddivide in tre passaggi principali:

1. Salvataggio (automatico) dell'*Exception Frame* e push manuale dei registri rimanenti (R4-R11).
2. Passaggio del MSP alla funzione *Capture_Registers* che si occupa di popolare la struttura *SystemState*.
3. Passaggio alla funzione *Save_State_EEPROM* che inizialmente calcola l'ingombro dello stack e successivamente salva concretamente i dati in *EEPROM*.

2.2 Persistenza su *EEPROM* e politica di validità

Il checkpoint è considerato valido se all'indirizzo *EEPROM* 0x08080000 è presente la *Magic Word* "0xACCEDED". Questo marcatore viene scritto all'avvio del salvataggio. Durante il boot, il firmware effettua una lettura diretta di questo indirizzo: in caso di corrispondenza avvia la procedura di ripristino, altrimenti procede con l'esecuzione nominale da zero. Al termine corretto del benchmark, la *Magic Word* viene sovrascritta con 0x00000000 (invalidazione) per impedire il ripristino di uno stato obsoleto ai boot successivi.

2.3 Procedura di restore

In fase di boot, se la *Magic Word* è valida, il firmware copia lo stato logico (struttura *SystemState* e stack dump) da *EEPROM* a RAM nei rispettivi indirizzi di origine, impiegando variabili statiche per non sovrascrivere lo stack durante il ripristino. Subito dopo viene richiamata la procedura naked *Restore_Context* in assembly, che compie il ripristino dei registri:

- Imposta il MSP al valore salvato
- Effettua il pop dei registri R4-R11
- Avanza l'MSP di 32 byte per ripulire l'*Exception Frame* hardware
- Imposta il Thumb bit (orr r2, r2, #1) sul PC salvato prima di spingerlo sullo stack
- Ripristina i flag aritmetici di stato in APSR e gli altri registri hardware
- Esegue il salto finale tramite pop {pc} all'istruzione successiva al checkpoint.

2.4 Benchmark applicativo (LU decomposition)

Il benchmark implementato è la decomposizione LU con risoluzione del sistema lineare associato, avente complessità $O(N^3)$ su matrice quadrata di dimensione 15x15. L'algoritmo calcola una matrice triangolare inferiore L e una superiore U tali che $A = L * U$, prevenendo divisioni per zero incrementando la diagonale principale. I checkpoint sono posizionati strategicamente:

- alla fine di ciascuna delle 15 iterazioni della decomposizione LU (dopo aver completato la riga/colonna di L e U).
- prima della Forward Substitution ($L * y = b$).
- prima della Backward Substitution ($U * x = y$).

Al completamento di ogni fase i risultati vengono stampati su porta seriale UART. La Figura 2.4.1 mostra l'output seriale relativo all'inizializzazione del benchmark e ai primi checkpoint, mentre la Figura 2.4.2 riporta la stampa finale delle soluzioni con la successiva cancellazione della firma in EEPROM.

```

===== SYSTEM INITIALIZATION =====

Stack status before the function:
Starting address: 0x20014000
Current MSP: 0x20013FD8
Stack dimension: 40 byte

No valid checkpoint found

===== STARTING LU DECOMPOSITION =====

Initializing matrix 15 x 15

Starting calculation:

Calculating [1 / 15]

Matrix at the end of the phase 1:

Lower matrix:
1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.16 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.31 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.46 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.60 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.75 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.90 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
1.05 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
1.20 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
1.35 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
1.50 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
1.64 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
1.79 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
1.94 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
2.09 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00

Upper matrix:
101.00 2.00 3.00 4.00 5.00 6.00 7.00 8.00 9.00 10.00 11.00 12.00 13.00 14.00 15.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00

Saving state in EEPROM (Checkpoint 1)

Starting EEPROM dump
Bytes to copy in EEPROM: 3040
Saving SystemState
Saving Local variables
EEPROM dump completed

```

Figura 2.4.1. Inizializzazione del sistema

```

===== SYSTEM SOLUTION =====

x[0] = 0.0476
x[1] = 0.0472
x[2] = 0.0468
x[3] = 0.0464
x[4] = 0.0459
x[5] = 0.0455
x[6] = 0.0451
x[7] = 0.0446
x[8] = 0.0442
x[9] = 0.0438
x[10] = 0.0433
x[11] = 0.0429
x[12] = 0.0425
x[13] = 0.0421
x[14] = 0.0416

===== FUNCTION COMPLETED =====

Magic word deleted

```

Figura 2.4.2. Soluzioni del sistema e cancellazione della firma in EEPROM

Figura 3.1.1. Reset tramite pulsante

[illegible]

Figura 3.1.2. Reset tramite interruzione di alimentazione

3.2 Reset con dump in *EEPROM* non completato

In questo scenario è stato introdotto un ritardo software per avere la possibilità di premere il pulsante prima del dump dei dati in *EEPROM*, in questa specifica situazione l'ultimo checkpoint valido salvato era quello della fase 14, il sistema ha eseguito la fase 15 ma non ha avuto il tempo di salvare i dati dato che è avvenuto un reset, perciò, è correttamente ripartito dal calcolo della fase 15. La Figura 3.2.1 mostra infatti che, dopo il reset avvenuto durante il dump, l'esecuzione riparte correttamente dalla fase 15.

```
Calculating [15 / 15]
Matrix at the end of the phase 15:
Lower matrix:
1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.16 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.31 0.27 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.46 0.39 0.31 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.60 0.52 0.41 0.31 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.75 0.65 0.51 0.38 0.29 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.90 0.77 0.60 0.45 0.34 0.26 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
1.05 0.90 0.70 0.52 0.39 0.30 0.23 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
1.20 1.03 0.80 0.60 0.44 0.34 0.26 0.20 1.00 0.00 0.00 0.00 0.00 0.00 0.00
1.35 1.15 0.89 0.67 0.50 0.37 0.29 0.22 0.18 1.00 0.00 0.00 0.00 0.00 0.00
1.50 1.28 0.99 0.74 0.55 0.41 0.32 0.25 0.19 0.16 1.00 0.00 0.00 0.00 0.00
1.64 1.40 1.09 0.81 0.60 0.45 0.34 0.27 0.21 0.17 0.14 1.00 0.00 0.00 0.00
1.79 1.53 1.18 0.88 0.65 0.49 0.37 0.29 0.23 0.18 0.14 0.12 1.00 0.00 0.00
1.94 1.66 1.28 0.95 0.71 0.53 0.40 0.31 0.24 0.19 0.15 0.12 0.10 1.00 0.00
2.09 1.78 1.38 1.03 0.76 0.57 0.43 0.33 0.26 0.20 0.16 0.13 0.10 0.08 1.00

Upper matrix:
101.00 2.00 3.00 4.00 5.00 6.00 7.00 8.00 9.00 10.00 11.00 12.00 13.00 14.00 15.00
0.00 116.68 17.52 18.37 19.21 20.05 20.89 21.73 22.57 23.42 24.26 25.10 25.94 26.78 27.62
0.00 0.00 127.37 27.83 28.30 28.77 29.23 29.70 30.17 30.63 31.10 31.57 32.03 32.50 32.97
0.00 0.00 0.00 131.25 31.31 31.38 31.45 31.51 31.58 31.65 31.71 31.78 31.85 31.91 31.98
0.00 0.00 0.00 0.00 130.71 30.46 30.20 29.95 29.70 29.44 29.19 28.93 28.68 28.43 28.17
0.00 0.00 0.00 0.00 0.00 128.25 27.77 27.28 26.80 26.31 25.83 25.34 24.85 24.37 23.88
0.00 0.00 0.00 0.00 0.00 0.00 125.24 24.59 23.94 23.29 22.63 21.98 21.33 20.68 20.03
0.00 0.00 0.00 0.00 0.00 0.00 0.00 122.24 21.47 20.70 19.92 19.15 18.38 17.60 16.83
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 119.46 18.59 17.72 16.85 15.98 15.11 14.24
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 116.93 15.98 15.02 14.07 13.12 12.17
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 114.62 13.59 12.57 11.54 10.51
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 112.49 11.39 10.29 9.19
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 110.49 9.31 8.14
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 108.56 7.31
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 106.66

Saving state in EEPROM (Checkpoint 15)

Starting EEPROM dump
Bytes to copy in EEPROM: 3040
Saving SystemState
Saving Local variables

Reset cause: Reset button (NRST)
===== SYSTEM INITIALIZATION =====
Stack status before the function:
Starting address: 0x20014000
Current MSP: 0x20013FD8
Stack dimension: 40 byte

Valid checkpoint found in EEPROM, starting recovery process

Calculating [15 / 15]
Matrix at the end of the phase 15:
Lower matrix:
1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.16 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.31 0.27 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.46 0.39 0.31 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.60 0.52 0.41 0.31 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.75 0.65 0.51 0.38 0.29 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.90 0.77 0.60 0.45 0.34 0.26 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
1.05 0.90 0.70 0.52 0.39 0.30 0.23 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
1.20 1.03 0.80 0.60 0.44 0.34 0.26 0.20 1.00 0.00 0.00 0.00 0.00 0.00 0.00
1.35 1.15 0.89 0.67 0.50 0.37 0.29 0.22 0.18 1.00 0.00 0.00 0.00 0.00 0.00
1.50 1.28 0.99 0.74 0.55 0.41 0.32 0.25 0.19 0.16 1.00 0.00 0.00 0.00 0.00
1.64 1.40 1.09 0.81 0.60 0.45 0.34 0.27 0.21 0.17 0.14 1.00 0.00 0.00 0.00
1.79 1.53 1.18 0.88 0.65 0.49 0.37 0.29 0.23 0.18 0.14 0.12 1.00 0.00 0.00
1.94 1.66 1.28 0.95 0.71 0.53 0.40 0.31 0.24 0.19 0.15 0.12 0.10 1.00 0.00
2.09 1.78 1.38 1.03 0.76 0.57 0.43 0.33 0.26 0.20 0.16 0.13 0.10 0.08 1.00
```

Figura 3.2.1. Reset prima del dump in *EEPROM*

3.3 Reset relativi alla fase di risoluzione

In questi scenari è stato testato il reset in due fasi distinte rispetto al ciclo visto nei casi precedenti: il primo è stato effettuato subito dopo la fase 15 (a dump completato questa volta), il sistema ha risposto correttamente riprendendo l'esecuzione dalla fase di risoluzione del sistema. Nel secondo scenario il pulsante è stato premuto tra le due fasi della risoluzione, in particolare subito la fase di Forward Substitution, il sistema è correttamente ripartito dalla fase di Backward Substitution. La Figura 3.3.1 documenta il reset tra la fase matriciale e la fase di risoluzione, mentre la Figura 3.3.2 mostra il reset tra Forward Substitution e Backward Substitution.

```
Calculating [15 / 15]
Matrix at the end of the phase 15:
Lower matrix:
1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.16 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.31 0.27 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.46 0.39 0.31 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.60 0.52 0.41 0.31 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.75 0.65 0.51 0.38 0.29 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.90 0.77 0.60 0.45 0.34 0.26 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
1.05 0.90 0.70 0.52 0.39 0.30 0.23 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
1.20 1.03 0.80 0.60 0.44 0.34 0.26 0.20 1.00 0.00 0.00 0.00 0.00 0.00 0.00
1.35 1.15 0.89 0.67 0.50 0.37 0.29 0.22 0.18 1.00 0.00 0.00 0.00 0.00 0.00
1.50 1.28 0.99 0.74 0.55 0.41 0.32 0.25 0.19 0.16 1.00 0.00 0.00 0.00 0.00
1.64 1.40 1.09 0.81 0.60 0.45 0.34 0.27 0.21 0.17 0.14 1.00 0.00 0.00 0.00
1.79 1.53 1.18 0.88 0.65 0.49 0.37 0.29 0.23 0.18 0.14 0.12 1.00 0.00 0.00
1.94 1.66 1.28 0.95 0.71 0.53 0.40 0.31 0.24 0.19 0.15 0.12 0.10 1.00 0.00
2.09 1.78 1.38 1.03 0.76 0.57 0.43 0.33 0.26 0.20 0.16 0.13 0.10 0.08 1.00

Upper matrix:
101.00 2.00 3.00 4.00 5.00 6.00 7.00 8.00 9.00 10.00 11.00 12.00 13.00 14.00 15.00
0.00 116.68 17.52 18.37 19.21 20.05 20.89 21.73 22.57 23.42 24.26 25.10 25.94 26.78 27.62
0.00 0.00 127.37 27.83 28.30 28.77 29.23 29.70 30.17 30.63 31.10 31.57 32.03 32.50 32.97
0.00 0.00 0.00 131.25 31.31 31.38 31.45 31.51 31.58 31.65 31.71 31.78 31.85 31.91 31.98
0.00 0.00 0.00 0.00 130.71 30.46 30.20 29.95 29.70 29.44 29.19 28.93 28.68 28.43 28.17
0.00 0.00 0.00 0.00 0.00 128.25 27.77 27.28 26.80 26.31 25.83 25.34 24.85 24.37 23.88
0.00 0.00 0.00 0.00 0.00 0.00 125.24 24.59 23.94 23.29 22.63 21.98 21.33 20.68 20.03
0.00 0.00 0.00 0.00 0.00 0.00 0.00 122.24 21.47 20.70 19.92 19.15 18.38 17.60 16.83
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 119.46 18.59 17.72 16.85 15.98 15.11 14.24
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 116.93 15.98 15.02 14.07 13.12 12.17
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 114.62 13.59 12.57 11.54 10.51
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 112.49 11.39 10.29 9.19
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 110.49 9.31 8.14
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 108.56 7.31
0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 106.66

Saving state in EEPROM (Checkpoint 15)

Starting EEPROM dump
Bytes to copy in EEPROM: 3040
Saving SystemState
Saving Local variables
EEPROM dump completed

Reset cause: Reset button (NRST)
===== SYSTEM INITIALIZATION =====

Stack status before the function:
Starting address: 0x20014000
Current MSP: 0x20013FD8
Stack dimension: 40 byte

Valid checkpoint found in EEPROM, starting recovery process
===== SYSTEM RESOLUTION =====

Right-hand side vector (b):
10.00
20.00
30.00
40.00
50.00
60.00
70.00
80.00
90.00
100.00
110.00
120.00
130.00
140.00
150.00
```

Figura 3.3.1. Reset tra fase matriciale e fase di risoluzione

```

Forward Substitution

Saving state before Backward substitution

Starting EEPROM dump
Bytes to copy in EEPROM: 3040
Saving SystemState
Saving Local variables
EEPROM dump completed

Reset cause: Reset button (NRST)
===== SYSTEM INITIALIZATION =====

Stack status before the function:
Starting address: 0x20014000
Current MSP:      0x20013FD8
Stack dimension:  40 byte

Valid checkpoint found in EEPROM, starting recovery process

Backward Substitution

```

Figura 3.3.2. Reset tra fase di Forward Substitution e fase di Backward Substitution

3.4 Corretta cancellazione della firma

In questo scenario l'obiettivo è quello di mostrare l'effettiva cancellazione della firma in *EEPROM*: alla fine dell'esecuzione del benchmark è stato premuto il tasto di reset e il sistema è correttamente ripartito dall'inizializzazione della matrice. La Figura 3.4 evidenzia il cold boot successivo alla cancellazione della Magic Word.

```

===== FUNCTION COMPLETED =====

Magic word deleted

Reset cause: Reset button (NRST)
===== SYSTEM INITIALIZATION =====

Stack status before the function:
Starting address: 0x20014000
Current MSP:      0x20013FD8
Stack dimension:  40 byte

No valid checkpoint found

===== STARTING LU DECOMPOSITION =====

Initializing matrix 15 x 15

Starting calculation:

Calculating [1 / 15]

```

Figura 3.4. Corretta cancellazione della firma e conseguente Cold Boot

4. Analisi delle prestazioni e ottimizzazione hardware

Per ottenere un prospetto temporale corretto dell'operazione di salvataggio in *EEPROM* (la più costosa sotto questo aspetto), si è sfruttato il registro hardware CYCCNT integrato nel modulo DWT del Cortex-M. La scelta è stata fatta principalmente perché il DWT opera misurando i singoli cicli di clock della CPU evitando conflitti di priorità con i segnali asincroni come il PendSV che si sarebbero potuti presentare con l'utilizzo di una libreria software standard come HAL_GetTick().

I risultati sperimentali (ottenuti con la CPU operante a 32MHz) dimostrano come il tempo di scrittura cresca con il susseguirsi delle fasi di elaborazione, questo comportamento è dovuto principalmente al fatto che la matrice iniziale sia quasi interamente composta da valori nulli mentre quella finale si compone di numeri (float) più complessi che quindi richiedono una maggiore elaborazione da parte della EEPROM.

In accordo con quanto detto sopra, si evidenziano i valori più significativi del processo: La Figura 4.1 riporta un esempio di output seriale utilizzato per stimare il tempo di dump.

Checkpoint misurato	Tempo di salvataggio effettivo	Osservazioni sulla memoria
Checkpoint 1	3571 ms and 722 μ s	Memoria RAM composta principalmente da zeri
Checkpoint 7	4037 ms and 962 μ s	Fase intermedia, sono presenti valori complessi e zeri in quantità simili
Checkpoint 15	4326 ms and 866 μ s	Worst-case, le matrici sono piene di valori complessi

```
Saving state in EEPROM (Checkpoint 15)
Starting EEPROM dump
Bytes to copy in EEPROM: 3040

CPU Clock Cycles Spent: 138459727
Effective Dump Time: 4326 ms and 866  $\mu$ s

EEPROM dump completed
```

Figura 4.1. Visualizzazione su terminale dei risultati di misurazione (per queste misurazioni sono stati rimossi i messaggi di stampa "Saving SystemState" e "Saving Local Variables" per ottenere una misurazione corretta)

L'analisi del worst-case, che evidenzia un tempo di esecuzione di circa 4.3 secondi, rappresenta il dato più critico per la progettazione elettronica del sistema. Un condensatore standard non possiede la capacità necessaria a mantenere in vita la CPU per oltre 4 secondi in assenza di alimentazione principale; pertanto, in un contesto reale sarebbe necessaria la presenza di un supercondensatore o di una batteria di backup, dimensionati correttamente per sostenere l'assorbimento di corrente per tutta la durata del dump in memoria non volatile.

5. Conclusioni e sviluppi futuri

Il prototipo sviluppato dimostra la fattibilità e l'efficacia del meccanismo di *fault tolerance* basato su Checkpoint/Restart per schede STM32 sottoposte a workload numerici impegnativi. L'implementazione in ambiente *bare-metal*, unita al recupero del contesto tramite la manipolazione diretta del Main Stack Pointer (MSP) in linguaggio Assembly, si è rivelata un metodo estremamente affidabile per garantire la corretta ripresa dell'esecuzione a fronte di reset arbitrari.

L'architettura è stata progettata per essere universale e agnostica, la routine di salvataggio in *EEPROM* esegue un salvataggio dello Stack indipendente dall'algoritmo in esecuzione nel main, in altre parole il sistema è applicabile a qualsiasi altro calcolo senza dover modificare il codice di salvataggio dei dati e ripristino del contesto.

Tuttavia, è giusto riconoscere che l'implementazione attuale presenta dei limiti concreti per una eventuale applicazione in scenari industriali safety-critical:

- **Efficienza e Incremental Checkpointing:** il limite principale dell'approccio universale risiede nelle tempistiche di scrittura dei dati in memoria, questo perché lo Stack viene continuamente sovrascritto indipendentemente dal fatto che i valori siano uguali ai precedenti per determinate righe e colonne. Per il benchmark scelto, che genera risultati definitivi ad ogni iterazione, si potrebbe pensare ad un meccanismo di Checkpoint incrementali che non andrebbe a sovrascrivere i dati già presenti ma soltanto quelli calcolati nella fase corrente; così facendo si andrebbe ad abbattere i tempi di scrittura, sacrificando però l'universalità dell'algoritmo.
- **Integrità dei dati:** nella versione attuale, se l'interruzione di alimentazione dovesse verificarsi durante il salvataggio dei dati in *EEPROM* (dopo che la *Magic Word* è già stata scritta), il sistema potrebbe tentare erroneamente di ripristinare il contesto che risulterebbe, almeno parzialmente, corrotto. Per ovviare a questo problema sarà necessario introdurre dei controlli di integrità dei dati (CRC/Checksum) uniti ad un'architettura Dual-Slot in cui i checkpoint vengono alternati in due aree di memoria distinte: con questa configurazione, il tentativo di ripristino di un checkpoint (n) i cui dati risultano corrotti non porterebbe ad una perdita di tutti i progressi ma riporterebbe soltanto all'ultimo checkpoint valido (n-1).
- **Variabili globali:** attualmente il sistema salva unicamente le variabili locali (stack), un'evoluzione di questo progetto in termini di utilità e complessità, richiederebbe la serializzazione delle sezioni *.data* e *.bss* (dove risiedono le variabili globali, sul "fondo" della RAM).